

Calcolatori Elettronici

20 febbraio 2024

Teoria

Esercizio 1 (12 punti): Progettare una rete sequenziale in grado di interpretare sequenze infinite di cifre espresse in codifica *Binary-coded decimal* (BCD). Ciascuna cifra viene inviata in input alla rete sequenziale un bit alla volta, partendo dal bit più significativo. Dopo aver letto una cifra BCD, la rete sequenziale fornisce in output il valore "sì" se la cifra letta è dispari, in tutti gli altri casi fornisce in output il valore "no". Realizzare il circuito equivalente dell'equazione minimizzata dell'uscita.

Esercizio 2 (12 punti): Si consideri il seguente frammento di codice:

```
1  movq $0x7, %rdi
2  testq %rdi, %rdi
3  jz .label
4  subq $0x1, %rsi
5  .label:
6  addq $0x2, %rdi
```

Discutere quali alee si verificano in un processore a pipeline a 5 stadi e come queste possono essere risolte dall'unità di controllo del processore.

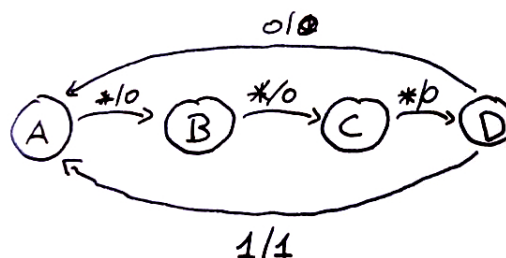
Esercizio 3 (6 punti): Si consideri il numero $A=1X.Y2$, dove X e Y sono rispettivamente il penultimo e l'ultimo numero della propria matricola. Convertire il numero in formato IEEE 754, mostrando tutti i passaggi.

Soluzioni

Esercizio 1

Il formato BCD utilizza 4 bit per codificare le cifre da 0 a 9: $(0)_{10} = (0000)_2$; $(1)_{10} = (0001)_2$; $(2)_{10} = (0010)_2$; $(3)_{10} = (0011)_2$; eccetera. Si nota quindi che se una cifra è dispari in decimale, l'ultimo dei 4 bit sarà 1. Solo in questo caso l'uscita dovrà essere "sì".

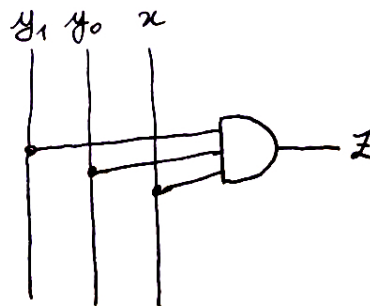
Pertanto, occorre realizzare una rete sequenziale che conti 4 bit e verifichi se l'ultimo dei 4 è 0 o 1. L'alfabeto di input è dato, per quello di output si utilizza l'ovvia codifica: sì = 1, no = 0, richiedendo quindi un solo bit z per rappresentare l'uscita. Volendo procedere con la progettazione di una macchina di Mealy, si ottiene il seguente automa:



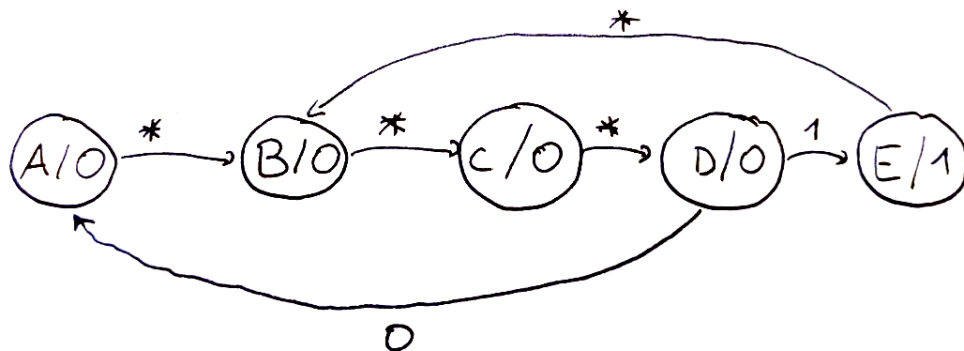
Utilizzando come codifica degli stati: $A=00$, $B=01$, $C=10$, $D=11$, si ottiene la seguente tabella per il calcolo della funzione di uscita:

y_1	y_0	x	z
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

La tabella è completamente specificata, quindi l'equazione dell'uscita è data direttamente dall'ultimo mintermine:
 $z = y_1 y_0 x$, che porta al circuito:



Se avessimo invece voluto utilizzare una macchina di Moore, avremmo ottenuto il seguente automa:



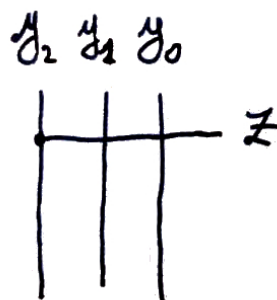
Adottando la stessa codifica degli stati precedente (utilizzando però 3 bit e codificando $E=100$), otteniamo la seguente tabella di calcolo della funzione d'uscita:

y_2	y_1	y_0	z
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	-
1	1	0	-
1	1	1	-

È importante notare che in questo caso la funzione di uscita non è completamente specificata, pertanto è possibile che sia possibile minimizzare la funzione. Procedendo con la mappa di Karnaugh a 3 variabili:

$y_2 y_1$	00	01	11	10
y_0				
0	0	0	-	1
1	0	0	-	-

otteniamo come funzione d'uscita $z = y_2$ (il che poteva anche essere immediatamente osservabile dalla tabella precedente). Si ottiene pertanto il circuito:



È molto interessante notare il tradeoff tra le realizzazioni con Mealy e Moore in questo caso: il calcolo dell'uscita con l'utilizzo di una macchina di Moore sarà molto più rapido, poiché non coinvolge alcuna porta logica (e quindi non ha ritardo). Tuttavia, è necessario utilizzare un bit di stato in più, richiedendo quindi un flip/flop aggiuntivo. Il trade-off velocità/consumo di memoria è una caratteristica tipica e ricorrente dei sistemi informatici, a vari livelli.

Esercizio 2

Sono possibili le seguenti alee nel codice proposto:

```
1  movq $0x7, %rdi
2  testq %rdi, %rdi # criticità sui dati per %rdi
3  jz .label # criticità sul controllo
4  subq $0x1, %rsi
5  .label:
6  addq $0x2, %rdi # Potenziale criticità sui dati per %rdi se si utilizza una branch prediction unit
```

Infatti:

- L'istruzione `testq %rdi, %rdi` effettua una lettura dopo una scrittura da parte dell'istruzione `movq $0x7, %rdi`, ma il valore di `%rdi` sarà scritto soltanto in uno stadio successivo.
- Se si utilizza una branch prediction unit, il processore potrebbe effettuare la previsione "salta" ed inserire in pipeline immediatamente l'istruzione `addq $0x2, %rdi`. Si osserva quindi nuovamente una dipendenza da `%rsi`. Tuttavia, se si risolve la criticità strutturale sul banco dei registri, questa criticità viene automaticamente risolta.
- L'istruzione di salto condizionale determina sempre una criticità strutturale sul controllo, poiché il valore del registro FLAGS viene aggiornato nello stadio di execute, ben successivo allo stadio di fetch. Non è corretto assumere che non ci sia la criticità sul controllo in questo caso, anche se il valore scritto in `%rdi` aggiornerà i flags in maniera da non far saltare il programma a `.label`: infatti, in fase di fetch, il processore non può sapere quale sarà il risultato dell'istruzione `testq %rdi, %rdi`.

Esercizio 3

Assumendo $x=1$ e $y=2$, per convertire 11.22 in IEEE 754, devo effettuare separatamente la conversione in decimale della parte intera e della parte frazionaria. Per la parte intera, per divisioni successive, si ottiene $(11)_{10} = (1011)_2$. La parte frazionaria, per moltiplicazioni successive, porta ad un numero periodico: $(0.22)_{10} = (0.0011100001010001111010)$.

Per effettuare la conversione, occorre normalizzare il numero, ottenendo:

$$1.0110011100001010001111010 \cdot 2^3.$$

L'esponente deve essere rappresentato in eccesso a 127, portando quindi a $3 + 127 = (130)_{10} = (10000010)_2$.

Poiché il numero è positivo, il segno è 0. La mantissa deve essere troncata a 23 bit, portando ad una perdita di precisione.

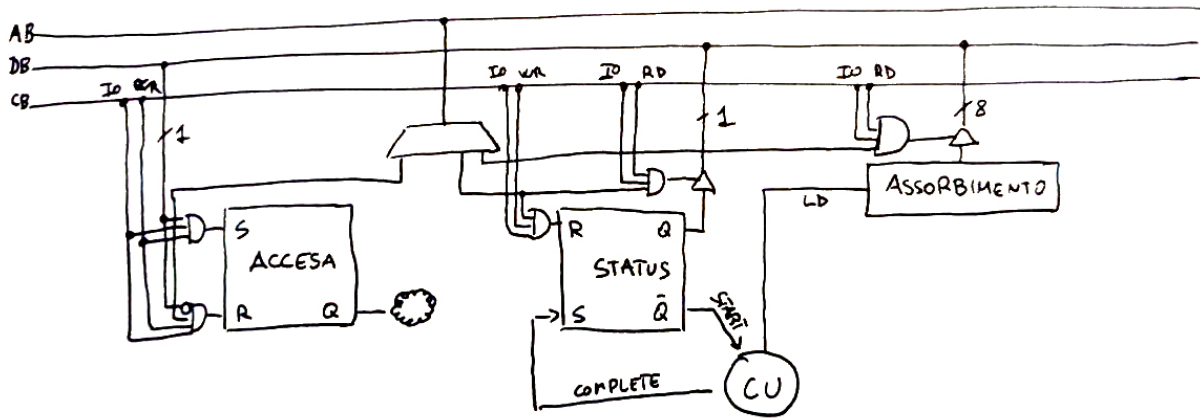
Si ottiene, in definitiva, la rappresentazione seguente: `0|10000010|01000001001100111000010100011110`

Progetto

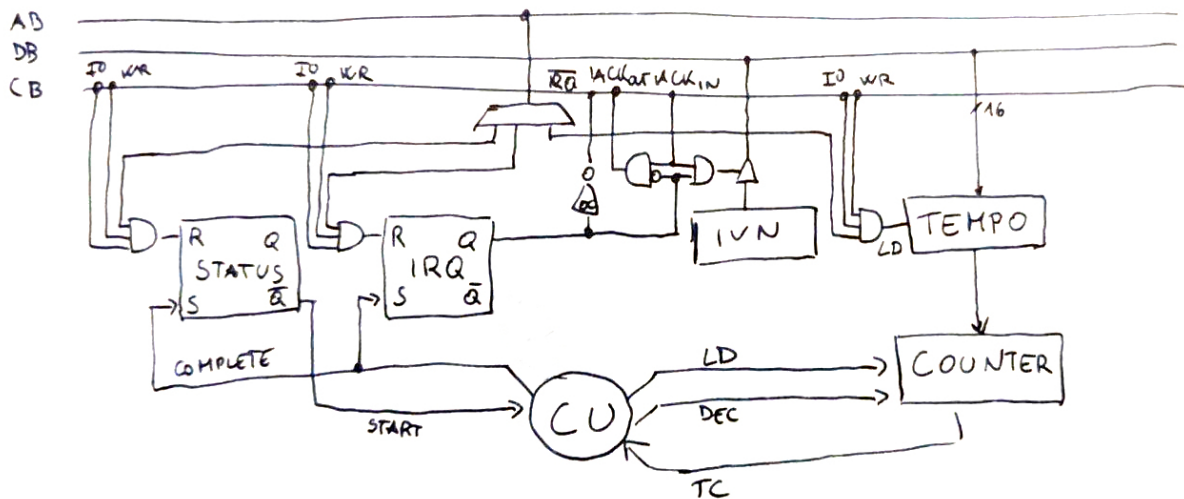
Un processore z64 controlla un piano cottura a induzione. Il piano è composto dalle periferiche `SELETTORE`, `BOBINA` e `TIMER`.

`SELETTORE` consente all'utilizzatore di indicare la potenza di cottura che si vuole utilizzare. È un dispositivo asincrono che, quando pilotato dall'utilizzatore, notifica al processore z64 un valore numerico nell'intervallo $[0, 9]$ indicante la potenza richiesta—0 corrisponde all'impianto spento.

La periferica **BOBINA** è sincrona, ma ha la particolarità di avere due modalità di interazione: nella prima, si può effettivamente accendere/spengere la bobina, nella seconda si può richiedere di produrre la potenza assorbita. Per la prima modalità operativa si utilizza il flip/flop **ACCESA** che opera come un interruttore. Per la seconda modalità operativa, si utilizza un regolare flip/flop di **STATUS** con cui implementare un protocollo di interazione in busy waiting.



La periferica **TIMER** lavora con le interruzioni. Dovendo essere programmabile, ha bisogno di un registro di interfaccia che consenta di scrivere dopo quanti millisecondi è necessario inviare la richiesta di interruzione.



La seguente è una sola delle molte possibili soluzioni dell'esercizio. Si è assunto che la bobina alterni periodi di accensione e spegnimento della stessa durata. Qualsiasi altra interpretazione è ugualmente valida.

```

1  .org 0x800
2  .data
3      .equ SELETTORE_IRQ, 0x1
4      .equ SELETTORE_POT, 0x2
5      .equ BOBINA_ACCESA, 0x3
6      .equ BOBINA_STATUS, 0x4
7      .equ BOBINA_ASSORB, 0x5
8      .equ TIMER_STATUS, 0x6
9      .equ TIMER_IRQ, 0x7
10     .equ TIMER_TEMPO, 0x8
11     .equ POTENZA_MINIMA, 20
12     temporizzazione: .word 1000, 881, 753, 620, 507, 374, 244, 122, 0
13     potenza: .byte 0
14     accesa: .byte 0
15     deve_essere_accesa: .byte 0
16 .text
17
18     main:
19         sti
20         .loop:

```

```

21     call verifica_assorbimento
22     cmpb $0, potenza
23     jz .loop
24     movb deve_essere_accesa, %al
25     cpmb %al, accesa
26     jz .loop
27     call commuta_bobina
28     jmp .loop
29
30 commuta_bobina:
31     movb deve_essere_accesa, %al
32     outb %al, $BOBINA_ACCESA
33     movb %al, accesa
34     cli
35     movb potenza, %al
36     cmpb $0, %al
37     jz .commutato
38     movzbq %al, %rax
39     movw temporizzazione(,%rax, 2), %ax
40     outw %ax, $TIMER_TEMPO
41     outb %al, $TIMER_STATUS
42 .commutato:
43     sti
44     ret
45
46 verifica_assorbimento:
47     outb %al, $BOBINA_STATUS
48 .bw:
49     inb $BOBINA_STATUS, %al
50     btb $0, %al
51     jnc .lbw
52     inb $BOBINA_ASSORB, %al
53     cmpb $POTENZA_MINIMA, %al
54     jc .ok2
55     call forza_spegnimento
56 .ok2:
57     ret
58
59 forza_spegnimento:
60     movb $0, %al
61     outb %al, $BOBINA_ACCESA
62     movb $0, accesa
63     movb $0, deve_essere_accesa
64     ret
65
66
67 .driver 0 # SELETTORE
68     push %rax
69     inb $SELETTORE_POT, %al
70     outb %al, $SELETTORE_IRQ
71     cmpb $0, %al
72     jnz .ok
73     call forza_spegnimento
74 .leave:
75     pop %rax
76     iret
77 .ok:
78     movb $1, deve_essere_accesa
79     jmp .leave
80
81 .driver 1 # TIMER
82     cmpb $0, potenza
83     jz .fine
84     addb $1, deve_essere_accesa
85     andb $1, deve_essere_accesa
86 .fine:

```

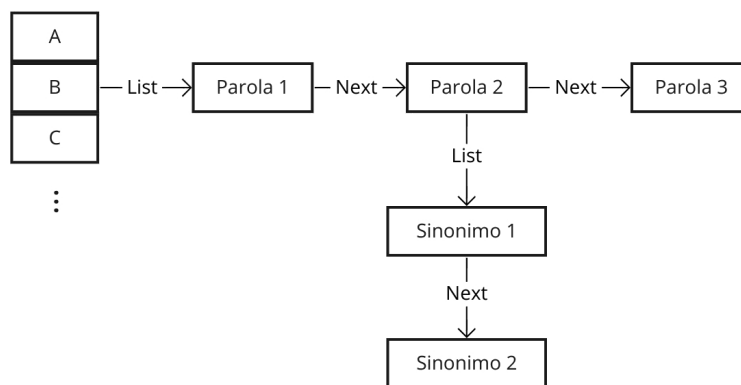
```
87     outb %aL, $TIMER_IRQ
88     iret
```

Prova di laboratorio

Si vuole realizzare un'applicazione in C in grado di compilare un dizionario dei sinonimi. Le operazioni che si chiede di supportare sono:

1. L'inserimento di una parola;
2. L'inserimento di un sinonimo di una parola;
3. La stampa di tutte le parole che iniziano con la stessa lettera;
4. Tutti i sinonimi di una parola inserita in input.

Il dizionario delle parole è organizzate in liste. Ciascuna lista contiene (in maniera non ordinata) tutte le parole che iniziano con la stessa lettera. Tali liste sono raggiungibili mediante un vettore. Data una parola, tutti i sinonimi sono anch'essi organizzati in una lista. L'organizzazione dei dati in memoria è riassunta dal seguente schema:



Per quanto riguarda l'operazione 2, se si tenta di inserire un sinonimo di una parola non presente nel dizionario, questa viene prima aggiunta al dizionario.

Soluzione

La seguente è una possibile soluzione all'esercizio. Si è scelto di mantenere tutte le parole in minuscolo, come in un vero dizionario.

```
1  #include <ctype.h>
2  #include <stdbool.h>
3  #include <stdio.h>
4  #include <stdlib.h>
5  #include <string.h>
6
7  #define BUFFER_SIZE 256
8
9  #define allocated_or_die(ptr) if(ptr == NULL) { \
10      fprintf(stderr, "Errore di allocazione memoria\n"); \
11      exit(EXIT_FAILURE); \
12  }
13
14  struct sinonimo {
```



```

15     char *sinonimo;
16     struct sinonimo *next;
17 };
18
19 struct parola {
20     char *parola;
21     struct sinonimo *sinonimi;
22     struct parola *next;
23 };
24
25 struct parola *dizionario[26] = {0};
26
27 struct parola *aggiungi_parola(char *parola)
28 {
29     int index = parola[0] - 'a';
30     struct parola *p = malloc(sizeof(struct parola));
31     allocated_or_die(p);
32     p->parola = strdup(parola);
33     allocated_or_die(p->parola);
34     p->next = dizionario[index];
35     dizionario[index] = p;
36     return p;
37 }
38
39 static struct parola *trova_parola(char *parola)
40 {
41     int index = parola[0] - 'a';
42     struct parola *p = dizionario[index];
43     while(p != NULL) {
44         if(strcmp(p->parola, parola) == 0) {
45             return p;
46         }
47         p = p->next;
48     }
49     return NULL;
50 }
51
52 void aggiungi_sinonimo(char *parola, char *sinonimo)
53 {
54     struct parola *p = trova_parola(parola);
55     if(p == NULL) {
56         p = aggiungi_parola(parola);
57     }
58
59     struct sinonimo *s = malloc(sizeof(struct sinonimo));
60     allocated_or_die(s);
61     s->sinonimo = strdup(sinonimo);
62     allocated_or_die(s->sinonimo);
63     s->next = p->sinonimi;
64     p->sinonimi = s;
65 }
66
67 void stampa_sinonimi(char *parola)
68 {
69     struct parola *p = trova_parola(parola);
70     if(p != NULL) {
71         printf("Parola trovata: %s\n", p->parola);
72         struct sinonimo *s = p->sinonimi;
73         while(s != NULL) {
74             printf("Sinonimo: %s\n", s->sinonimo);
75             s = s->next;
76         }
77     } else {
78         printf("Parola non trovata\n");
79     }
80 }

```

```

81
82 void stampa_parole_che_iniziano_per(char c)
83 {
84     int index = c - 'a';
85     struct parola *p = dizionario[index];
86     while(p != NULL) {
87         printf("%s\n", p->parola);
88         p = p->next;
89     }
90 }
91
92 void leggi_stringa(char *buffer, int size)
93 {
94     int c;
95     while(true) {
96         if(!fgets(buffer, size, stdin)) {
97             *buffer = '\0';
98             return;
99         }
100         if(buffer[0] != '\n')
101             break;
102     }
103
104     size_t len = strlen(buffer);
105     if(len > 0 && buffer[len - 1] == '\n')
106         buffer[len - 1] = '\0';
107
108     if(len == size - 1 && buffer[len - 1] != '\0')
109         while((c = getchar()) != '\n' && c != EOF);
110
111     for(int i = 0; buffer[i] != '\0'; i++)
112         buffer[i] = tolower(buffer[i]);
113 }
114
115 int main(void)
116 {
117     char buffer[BUFFER_SIZE];
118     char buffer2[BUFFER_SIZE];
119     while(true) {
120         printf("1. Aggiungi parola \n");
121         printf("2. Aggiungi sinonimo \n");
122         printf("3. Stampa sinonimi \n");
123         printf("4. Stampa parole che iniziano per \n");
124         printf("5. Esci\n");
125
126         leggi_stringa(buffer, 2);
127
128         switch(buffer[0]) {
129             case '1':
130                 printf("Inserisci la parola: ");
131                 leggi_stringa(buffer, BUFFER_SIZE);
132                 aggiungi_parola(buffer);
133                 break;
134             case '2':
135                 printf("Inserisci la parola: ");
136                 leggi_stringa(buffer, BUFFER_SIZE);
137                 printf("Inserisci il sinonimo: ");
138                 leggi_stringa(buffer2, BUFFER_SIZE);
139                 aggiungi_sinonimo(buffer, buffer2);
140                 aggiungi_sinonimo(buffer2, buffer); // I sinonimi sono simmetrici
141                 break;
142             case '3':
143                 printf("Inserisci la parola: ");
144                 leggi_stringa(buffer, BUFFER_SIZE);
145                 stampa_sinonimi(buffer);
146                 break;

```

```
147         case '4':
148             printf("Inserisci la lettera: ");
149             leggi_stringa(buffer, BUFFER_SIZE);
150             stampa_parole_che_iniziano_per(buffer[0]);
151             break;
152         case '5':
153             return 0;
154     }
155 }
156 }
```

